Example 4

The following 12-bit binary number represents denary number 1676:

−2048	1024	512	256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	0	0	1	1	0	0

Applying two's complement (1 0 0 1 0 1 1 1 0 0 1 1 + 1) would give:

−2048	1024	512	256	128	64	32	16	8	4	2	1
1	0	0	1	0	1	1	1	0	1	0	0

In other words: −1676

Activity 1.13

Convert the following negative denary numbers into binary numbers using the two's complement format:

a −18 **b** −31 **c** −47 **d** −63 **e** −88 **f** −92 **g** −100 **h** −1 **i** −16 **j** −127

Activity 1.14

Convert the following negative binary numbers (written in two's complement format) into negative denary numbers:

a	1	1	0	0	1	1	0	1
b	1	0	1	1	1	1	1	0
c	1	1	1	0	1	1	1	1
d	1	0	0	0	0	1	1	1
e	1	0	1	0	0	0	0	0
f	1	1	1	1	1	0	0	1
g	1	0	1	0	1	1	1	1
h	1	1	1	1	1	1	1	1
i	1	0	0	0	0	0	0	1
j	1	1	1	1	0	1	1	0

1.2 Text, sound and images

1.2.1 Character sets – ASCII code and Unicode

The **ASCII code** system (American Standard Code for Information Interchange) was set up in 1963 for use in communication systems and computer systems. A newer version of the code was published in 1986. The standard ASCII code **character set** consists of 7-bit codes (0 to 127 in denary or 00 to 7F in

hexadecimal) that represent the letters, numbers and characters found on a standard keyboard, together with 32 control codes (that use codes 0 to 31 (denary) or 00 to 19 (hexadecimal)).

Table 1.2 shows part of the standard ASCII code table (only the control codes have been removed).

▼ **Table 1.2** Part of the ASCII code table

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
32	20	<SPACE>	64	40	@	96	60	`
33	21	!	65	41	A	97	61	a
34	22	"	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	'	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2A	*	74	4A	J	106	6A	j
43	2B	+	75	4B	K	107	6B	k
44	2C	,	76	4C	L	108	6C	l
45	2D	-	77	4D	M	109	6D	m
46	2E	.	78	4E	N	110	6E	n
47	2F	/	79	4F	O	111	6F	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3A	:	90	5A	Z	122	7A	z
59	3B	;	91	5B	[	123	7B	{
60	3C	<	92	5C	\	124	7C	|
61	3D	=	93	5D	]	125	7D	}
62	3E	>	94	5E	^	126	7E	~
63	3F	?	95	5F	_	127	7F	<DELETE>

Consider the uppercase and lowercase codes in binary of characters. For example,

'a'	1	1	0	0	0	0	1	hex 61 (lower case)
'A'	1	0	0	0	0	0	1	hex 41 (upper case)
'y'	1	1	1	1	0	0	1	hex 79 (lower case)
'Y'	1	0	1	1	0	0	1	hex 59 (upper case)

The above examples show that the sixth bit changes from 1 to 0 when comparing the lowercase and uppercase of a character. This makes the conversion between the two an easy operation. It is also noticeable that the character sets (e.g. a to z, 0 to 9, etc.) are grouped together in sequence, which speeds up usability.

Extended ASCII uses 8-bit codes (0 to 255 in denary or 0 to FF in hexadecimal). This gives another 128 codes to allow for characters in non-English alphabets and for some graphical characters to be included:

DOS	WIN	Dec	Hex	DOS	WIN	Dec	Hex	DOS	WIN	Dec	Hex	DOS	WIN	Dec	Hex
Ç	€	128	80	á		160	A0	└	À	192	C0	α	à	224	E0
ü		129	81	í	¡	161	A1	┴	Á	193	C1	ß	á	225	E1
é	‚	130	82	ó	¢	162	A2	┬	Â	194	C2	Γ	â	226	E2
â	ƒ	131	83	ú	£	163	A3	├	Ã	195	C3	π	ã	227	E3
ä	„	132	84	ñ	¤	164	A4	─	Ä	196	C4	Σ	ä	228	E4
à	…	133	85	Ñ	¥	165	A5	┼	Å	197	C5	σ	å	229	E5
å	†	134	86	ª	¦	166	A6	╞	Æ	198	C6	µ	æ	230	E6
ç	‡	135	87	º	§	167	A7	╟	Ç	199	C7	τ	ç	231	E7
ê	ˆ	136	88	¿	¨	168	A8	╚	È	200	C8	Φ	è	232	E8
ë	‰	137	89	⌐	©	169	A9	╔	É	201	C9	Θ	é	233	E9
è	Š	138	8A	¬	ª	170	AA	╩	Ê	202	CA	Ω	ê	234	EA
ï	‹	139	8B	½	«	171	AB	╦	Ë	203	CB	δ	ë	235	EB
î	Œ	140	8C	¼	¬	172	AC	╠	Ì	204	CC	∞	ì	236	EC
ì		141	8D	¡	-	173	AD	═	Í	205	CD	ø	í	237	ED
Ä	Ž	142	8E	«	®	174	AE	╬	Î	206	CE	ε	î	238	EE
Å		143	8F	»	¯	175	AF	╧	Ï	207	CF	∩	ï	239	EF
É		144	90	░	°	176	B0	╨	Đ	208	D0	≡	ð	240	F0
æ	‘	145	91	▒	±	177	B1	╤	Ñ	209	D1	±	ñ	241	F1
Æ	’	146	92	▓	²	178	B2	╥	Ò	210	D2	≥	ò	242	F2
ô	“	147	93	│	³	179	B3	╙	Ó	211	D3	≤	ó	243	F3
ö	”	148	94	┤	´	180	B4	╘	Ô	212	D4	⌠	ô	244	F4
ò	•	149	95	╡	µ	181	B5	╒	Õ	213	D5	⌡	õ	245	F5
û	–	150	96	╢	¶	182	B6	╓	Ö	214	D6	÷	ö	246	F6
ù	—	151	97	╖	·	183	B7	╫	×	215	D7	≈	÷	247	F7
ÿ	˜	152	98	╕	¸	184	B8	╪	Ø	216	D8	°	ø	248	F8
Ö	™	153	99	╣	¹	185	B9	┘	Ù	217	D9	•	ù	249	F9
Ü	š	154	9A	║	º	186	BA	┌	Ú	218	DA	·	ú	250	FA
¢	›	155	9B	╗	»	187	BB	▉	Û	219	DB	√	û	251	FB
£	œ	156	9C	╝	¼	188	BC	▄	Ü	220	DC	ⁿ	ü	252	FC
¥		157	9D	╜	½	189	BD	▌	Ý	221	DD	²	ý	253	FD
Pts	ž	158	9E	╛	¾	190	BE	▐	Þ	222	DE	■	þ	254	FE
ƒ	Ÿ	159	9F	┐	¿	191	BF	▀	ß	223	DF		ÿ	255	FF

Figure 1.6 Extended ASCII code table

ASCII code has a number of disadvantages. The main disadvantage is that it does not represent characters in non-Western languages, for example Chinese characters. As can be seen in Figure 1.6 where DOS and Windows use different characters for some ASCII codes. For this reason, different methods of coding have been developed over the years. One coding system is called **Unicode**. Unicode can represent all languages of the world, thus supporting many operating systems, search engines and internet browsers used globally. There is overlap with standard ASCII code, since the first 128 (English) characters are the same, but Unicode can support several thousand different characters in total. As can be seen in Table 1.2 and Figure 1.6, ASCII uses one byte to represent a character, whereas Unicode will support up to four bytes per character.

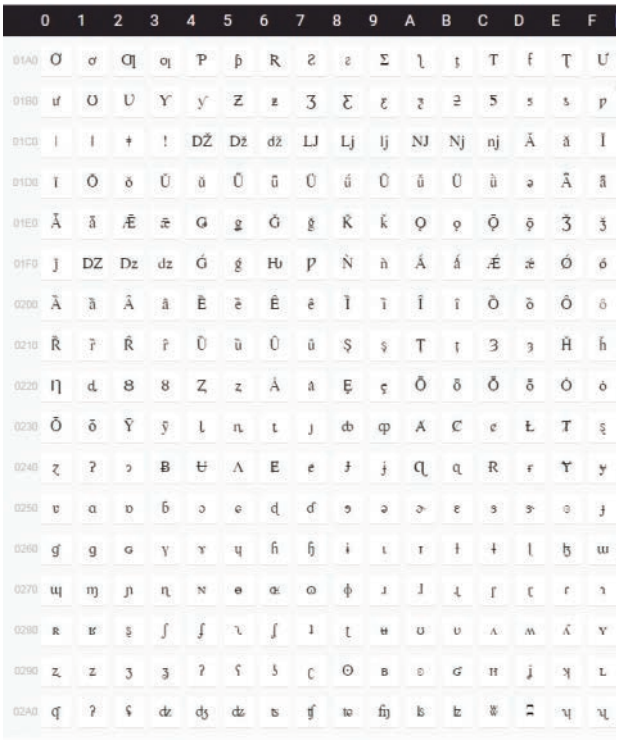
The Unicode consortium was set up in 1991. Version 1.0 was published with five goals; these were to:

- create a universal standard that covered all languages and all writing systems
- produce a more efficient coding system than ASCII
- adopt uniform encoding where each character is encoded as 16-bit or 32-bit code
- create unambiguous encoding where each 16-bit and 32-bit value always represents the same character
- reserve part of the code for private use to enable a user to assign codes for their own characters and symbols (useful for Chinese and Japanese character sets, for example).

Find out more

DOS appears in the ASCII extended code table. Find out what is meant by DOS and why it needs to have an ASCII code value.

A sample of Unicode characters are shown in Figure 1.7. As can be seen from the figure, characters used in languages such as Russian, Romanian and Croatian can now be represented in a computer).

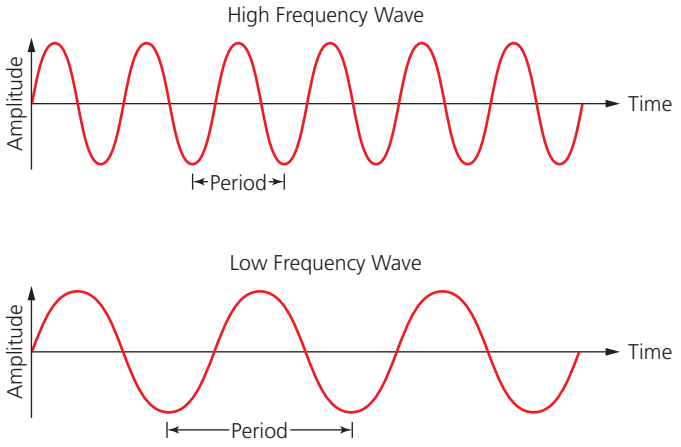


▲ **Figure 1.7** Sample of Unicode characters

1.2.2 Representation of sound

Soundwaves are vibrations in the air. The human ear senses these vibrations and interprets them as sound.

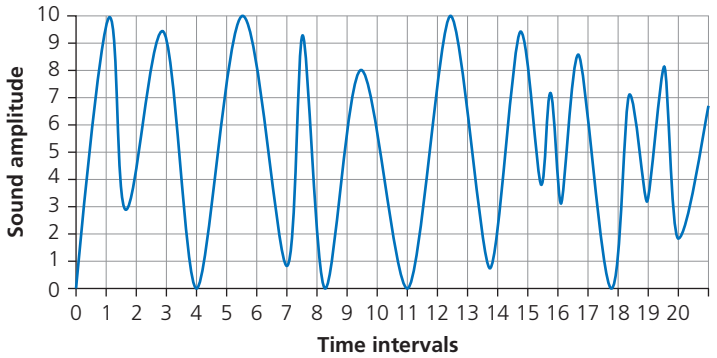
Each sound wave has a frequency, wavelength and amplitude. The amplitude specifies the loudness of the sound.



▲ **Figure 1.8** High and low frequency wave signals

Sound waves vary continuously. This means that sound is analogue. Computers cannot work with analogue data, so sound waves need to be sampled in order to be stored in a computer. Sampling means measuring the amplitude of the sound wave. This is done using an analogue to digital converter (ADC).

To convert the analogue data to digital, the sound waves are sampled at regular time intervals. The amplitude of the sound cannot be measured precisely, so approximate values are stored.



▲ **Figure 1.9** A sound wave being sampled

Figure 1.9 shows a sound wave. The x-axis shows the time intervals when the sound was sampled (1 to 21), and the y-axis shows the amplitude of the sampled sound to 10.

At time interval 1, the approximate amplitude is 10; at time interval 2, the approximate amplitude is 4, and so on for all 20 time intervals. Because the amplitude range in Figure 1.9 is 0 to 10, then 4 binary bits can be used to represent each amplitude value (for example, 9 would be represented by the

binary value 1001). Increasing the number of possible values used to represent sound amplitude also increases the accuracy of the sampled sound (for example, using a range of 0 to 127 gives a much more accurate representation of the sound sample than using a range of, for example, 0 to 10). The number of bits per sample is known as the **sampling resolution** (also known as the **bit depth**). So, in our example, the sampling resolution is 4 bits.

Sampling rate is the number of sound samples taken per second. This is measured in hertz (Hz), where 1 Hz means 'one sample per second'.

So how is sampling used to record a sound clip?

- the amplitude of the sound wave is first determined at set time intervals (the sampling rate)
- this gives an approximate representation of the sound wave
- each sample of the sound wave is then encoded as a series of binary digits.

Using a higher sampling rate or larger resolution will result in a more faithful representation of the original sound source. However, the higher the sampling rate and/or sampling resolution, the greater the file size.

▼ **Table 1.3** The benefits and drawbacks of using a larger sampling resolution when recording sound

Benefits	Drawbacks
larger dynamic range	produces larger file size
better sound quality	takes longer to transmit/download music files
less sound distortion	requires greater processing power

Link

See Section 1.3 for a calculation of file sizes.

CDs have a 16-bit sampling resolution and a 44.1 kHz sample rate – that is 44 100 samples every second. This gives high-quality sound reproduction.

1.2.3 Representation of (bitmap) images

Bitmap images are made up of **pixels** (picture elements); an image is made up of a two-dimensional matrix of pixels. Pixels can take different shapes such as:



▲ **Figure 1.10**

Each pixel can be represented as a binary number, and so a bitmap image is stored in a computer as a series of binary numbers, so that:

- a black and white image only requires 1 bit per pixel – this means that each pixel can be one of two colours, corresponding to either 1 or 0
- if each pixel is represented by 2 bits, then each pixel can be one of four colours ($2^2 = 4$), corresponding to 00, 01, 10, or 11
- if each pixel is represented by 3 bits then each pixel can be one of eight colours ($2^3 = 8$), corresponding to 000, 001, 010, 011, 100, 101, 110, 111.

The number of bits used to represent each colour is called the **colour depth**. An 8 bit colour depth means that each pixel can be one of 256 colours (because

$2^8 = 256$). Modern computers have a 24 bit colour depth, which means over 16 million different colours can be represented With x pixels, 2^x colours can be represented as a generalisation. Increasing colour depth also increases the size of the file when storing an image.

Image resolution refers to the number of pixels that make up an image; for example, an image could contain 4096 × 3072 pixels (12 582 912 pixels in total).

The resolution can be varied on many cameras before taking, for example, a digital photograph. Photographs with a lower resolution have less detail than those with a higher resolution. For example, look at Figure 1.11:



▲ **Figure 1.11** Five images of the same car wheel using different resolutions

Image 'A' has the highest resolution and 'E' has the lowest resolution. 'E' has become pixelated ('fuzzy'). This is because there are fewer pixels in 'E' to represent the image.

The main drawback of using high resolution images is the increase in file size. As the number of pixels used to represent the image is increased, the size of the file will also increase. This impacts on how many images can be stored on, for example, a hard drive. It also impacts on the time to download an image from the internet or the time to transfer images from device to device. A certain amount of reduction in resolution of an image is possible before the loss of quality becomes noticeable.

Activity 1.15

1 Explain each of the following terms:
   - i colour depth
   - ii ASCII code and Extended ASCII code
   - iii Unicode
   - iv sampling rate
   - v bitmap image

2 A colour image is made up of red, green and blue colour combinations. 8 bits are used to represent each of the colour components.
   - i How many possible variations of red are there?
   - ii How many possible variations of green are there?
   - iii How many possible variations of blue are there?
   - iv How many different colours can be made by varying the red, green and blue values?

3 Describe the effect of increasing resolution and sampling rate on the size of a file being stored in a computer.